Handwritten Digit Recognition using MNIST Dataset

Mostafa Badawe Fouad
ID: 2023019029
AI3

Contents

1 Project Description **2**

2 Dataset Information **2**

3 Data Preprocessing **2**
3.1 Converting Images to Tensors 2
3.2 Normalization 3

4 Dataset Split **3**

5 Neural Network Architecture **3**

6 Forward Propagation **4**

7 Activation Functions **4**
7.1 Experiment 1 – ReLU 4
7.2 Experiment 2 – Tanh 4

8 Loss Function **4**

9 Optimizer **5**

10 Training Process **5**

11 Accuracy Formula **5**

12 Experiment Results **6**

13 Experiment Analysis **6**

14 Final Conclusion **6**

1 Project Description

The goal of this project is to build a Neural Network model using PyTorch to classify handwritten digits from the MNIST dataset.

The model is trained to recognize digits from:

$$0 \rightarrow 9$$

A Multilayer Perceptron (MLP) architecture was implemented and evaluated using different experiments.

2 Dataset Information

The project uses the MNIST dataset provided by PyTorch using:

`torchvision.datasets.MNIST`

The MNIST dataset contains:

- 60,000 training images
- 10,000 testing images
- Grayscale handwritten digit images
- Image size: 28×28

3 Data Preprocessing

The MNIST dataset does not contain missing values, therefore no missing value handling was required.

The following preprocessing steps were applied before training the neural network.

3.1 Converting Images to Tensors

MNIST images were converted into PyTorch tensors using:

`transforms.ToTensor()`

Each image has a size of:

$$28 \times 28 \text{ pixels}$$

After flattening:

$$28 \times 28 = 784$$

Therefore, each image becomes a vector of size:

$$784$$

3.2 Normalization

Pixel values were normalized using:

```
transforms.Normalize((0.1307,), (0.3081,))
```

Normalization formula:

$$x_{normalized} = \frac{x - \mu}{\sigma}$$

Where:

- x = original pixel value
- $\mu = 0.1307$
- $\sigma = 0.3081$

Normalization improves training stability and helps the model converge faster.

4 Dataset Split

The dataset was divided into:

Dataset	Percentage
Training Set	80%
Validation Set	20%
Testing Set	Official MNIST Test Set

Table 1: Dataset Split

5 Neural Network Architecture

A simple Multilayer Perceptron (MLP) was implemented using PyTorch.
The network architecture consists of:

Layer	Size
Input Layer	784
Hidden Layer	128 / 256
Output Layer	10

Table 2: MLP Architecture

The output layer contains 10 neurons representing digits from 0 to 9.

6 Forward Propagation

During forward propagation, the output of each neuron is calculated using:

$$z = Wx + b$$

Where:

- W = weights
- x = input features
- b = bias

The activation function is then applied.

7 Activation Functions

7.1 Experiment 1 – ReLU

The ReLU activation function is defined as:

$$f(x) = \max(0, x)$$

Advantages of ReLU:

- Faster training
- Reduces vanishing gradient problem
- Efficient for image classification tasks

7.2 Experiment 2 – Tanh

The Tanh activation function is defined as:

$$\tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$$

The output range of Tanh is between:

$$-1 \text{ and } 1$$

8 Loss Function

The project used:

`CrossEntropyLoss()`

Cross-Entropy Loss formula:

$$L = -\sum_{i=1}^{C} y_i \log(\hat{y}_i)$$

Where:

- y_i = true label
- $\hat{y}_i$ = predicted probability
- C = number of classes

The objective of training is to minimize the loss value.

9 Optimizer

The Adam optimizer was used for updating network weights.

Weight update equation:

$$\theta = \theta - \alpha \nabla J(\theta)$$

Where:

- θ = model parameters
- α = learning rate
- $\nabla J(\theta)$ = gradient of the loss function

10 Training Process

The model was trained for:

5 Epochs

During training:

1. Images are passed through the neural network
2. Predictions are generated
3. Loss is calculated
4. Backpropagation updates the weights
5. Accuracy and loss are monitored

11 Accuracy Formula

Model accuracy was calculated using:

$$Accuracy = \frac{Correct\ Predictions}{Total\ Predictions} \times 100$$

12 Experiment Results

Experiment	Activation	Hidden Neurons	Learning Rate	Test Accuracy
Experiment 1	ReLU	128	0.001	97.18%
Experiment 2	Tanh	256	0.01	91.43%

Table 3: Experiment Comparison

13 Experiment Analysis

Experiment 1 achieved better performance than Experiment 2.

From the accuracy graph:

- Experiment 1 showed stable and continuous improvement during training.
- Experiment 2 showed fluctuating accuracy values, indicating unstable learning.

From the loss graph:

- Experiment 1 had lower training and validation loss.
- Experiment 2 had higher and unstable validation loss.

This indicates that ReLU activation with a smaller learning rate helped the model learn more effectively.

14 Final Conclusion

This project demonstrated how a simple Multilayer Perceptron (MLP) can successfully classify handwritten digits using the MNIST dataset.

The project also showed how changing:

- Activation functions
- Number of neurons
- Learning rate

can significantly affect neural network performance.

Experiment 1 achieved the best performance with:

97.18% Test Accuracy

which demonstrates that the model learned the handwritten digit classification task successfully.